哈尔滨工业大学（深圳）

实验报告

开课学期： 2026 年春季学期

课程名称： 软件构造

实验名称： 飞机大战系统设计与设计模式应用分析

实验性质： 设计型

实验学时： 24 地点：实验室

学生班级： 计算机与电子通信 12 班

学生学号： 2024311234

学生姓名： 何文杰

评阅教师：

报告成绩：

实验与创新实践教育中心制

2026 年 3 月

目录

1 实验环境 **3**

2 实验过程 **4**

2.1 项目需求与功能分析 4

2.2 项目结构分析 4

2.3 核心运行流程 5

2.4 UML 建模与分析 6

3 设计模式应用 **9**

3.1 单例模式 9

3.1.1 应用场景分析 9

3.1.2 设计模式结构图 10

3.2 简单工厂模式 11

3.2.1 应用场景分析 11

3.2.2 设计模式结构图 11

3.3 工厂模式 12

3.3.1 应用场景分析 12

3.3.2 设计模式结构图 13

3.4 策略模式 15

3.4.1 应用场景分析 15

3.4.2 设计模式结构图 15

3.5 数据访问对象模式 16

3.5.1 应用场景分析 16

3.5.2 设计模式结构图 17

3.6 观察者模式 18

3.6.1 应用场景分析 18

3.6.2 设计模式结构图 19

3.7 模板模式 20

3.7.1 应用场景分析 20

3.7.2 设计模式结构图 21

4 系统亮点与创新 **22**

4.1 多设计模式组合 22

4.2 弹幕系统具有较强扩展性 22

4.3 难度系统层次清晰 23
4.4 排行榜实现简单可靠 23
4.5 游戏反馈完整 23

5 收获与反思 **23**
5.1 核心收获 23
5.2 遇到的问题与解决方案 24
5.2.1 问题一：游戏主循环容易变成上帝类 24
5.2.2 问题二：弹幕逻辑变化频繁 24
5.2.3 问题三：不同难度之间重复代码较多 24
5.2.4 问题四：排行榜持久化与游戏逻辑耦合 24
5.3 不足与改进方向 24

6 参考资料 **25**

A 附录：总体架构 UML 类图 **25**

1 实验环境

本项目是一个基于 Java 与 Gradle 构建的桌面弹幕射击游戏。项目主要使用 Swing 完成窗口、菜单、游戏画面与排行榜界面；使用面向对象方式组织英雄机、敌机、子弹、道具、难度、音频、排行榜等模块；使用多种设计模式完成系统解耦与扩展。

实验开发与分析环境如下：

环境项目	说明
操作系统	Linux / Windows / macOS 均可运行，本报告分析环境以 Linux 开发环境为参考。
JDK 版本	JDK 25。项目 README 中将 JDK 25 列为运行要求。
构建工具	Gradle Wrapper，仓库中包含 `gradlew`、`gradlew.bat` 与 `gradle/` 目录。
主要语言	Java。
GUI 框架	Java Swing / AWT。主要使用 `JFrame`、`JPanel`、`Timer`、`Graphics2D` 等组件。
UML 工具	PlantUML，可使用 VS Code PlantUML 插件、IntelliJ IDEA PlantUML 插件或在线 PlantUML 渲染器。
测试工具	Gradle Test，运行命令为 `./gradlew test`。
运行命令	`./gradlew :apps:run`。
项目入口	`com.touhou.Main`。

项目推荐运行命令如下：

Listing 1: 项目构建、测试与运行命令

```
# 构建项目
./gradlew build

# 运行测试
./gradlew test

# 启动游戏
./gradlew :apps:run
```

2 实验过程

2.1 项目需求与功能分析

本项目实现的是一个“飞机大战”类型的弹幕射击游戏。玩家通过鼠标控制英雄机移动，英雄机会自动向上方发射子弹；敌机会从屏幕上方生成并向下移动，同时根据自身类型发射不同形态的弹幕。玩家子弹命中敌机会造成伤害，敌人死亡后给予分数并有概率掉落道具。玩家可拾取生命、火力强化、超级火力、炸弹、冻结等道具。游戏还支持普通敌机、精英敌机、Boss、不同难度、背景音乐、音效开关和本地排行榜。

从运行效果看，系统最终产出是一个完整的 Java Swing 桌面小游戏，包含如下功能：

1. 图形化菜单：支持玩家名输入、难度选择、声音开关、开始游戏和查看排行榜。
2. 游戏主循环：以固定帧率推进角色移动、敌人生成、子弹飞行、碰撞检测和界面刷新。
3. 多种敌人：包括普通敌人、精英敌人、强化精英敌人、王牌敌人和 Boss。
4. 多种道具：包括回血、火力强化、超级火力、炸弹与冻结。
5. 多种射击策略：包括单发、散射、环形、扇形、直线阵列等弹幕。
6. 多难度机制：简单、普通、困难三类难度具有不同刷怪、敌人强度、Boss 规则与成长机制。
7. 排行榜持久化：通过本地 TSV 文件保存不同难度下的玩家分数。

2.2 项目结构分析

项目源码主要位于 `apps/src/main/java/com/touhou`。根据包名和职责可以划分为如下层次：

包名	主要职责
`com.touhou`	程序入口，负责启动应用。
`ui`	用户界面层，负责菜单、排行榜、游戏画面绘制和窗口切换。
`core`	游戏核心逻辑层，负责主循环、角色更新、碰撞检测、上下文对象。
`components`	游戏实体层，定义英雄机、敌机、子弹、道具等核心对象。

strategy	射击策略层，定义不同弹幕生成算法。
factories	工厂层，负责创建敌人与道具对象。
difficulty	难度层，使用模板方法控制不同难度下的游戏节奏。
observer	观察者层，处理道具强化事件的广播与响应。
leaderboard	数据访问层，负责排行榜记录的存储、读取、排序与删除。
audio	音频层，负责背景音乐、音效播放和静音控制。

这种分层方式使项目基本符合“表示层、业务逻辑层、领域对象层、数据访问层”分离的思想。其中，Game 是核心协调者，CharacterSystem 负责游戏对象推进，CollisionSystem 负责碰撞结算，GameUI 负责渲染，DifficultyTemplate 负责难度规则。

2.3 核心运行流程

游戏启动后，Main 创建窗口和控制器，AppController 展示菜单界面。当用户选择难度并开始游戏后，控制器创建 Game 实例并将其加入窗口。游戏内部使用 Swing Timer 以固定频率调用 tick() 方法。每次 tick() 可以理解为一帧逻辑更新。

游戏主循环流程如下：

1. Game.tick() 增加当前帧计数。
2. 调用 DifficultyTemplate.action(context)，进入统一的难度模板流程。
3. 模板方法先调整玩家和敌人的难度相关参数。
4. CharacterSystem.onTick() 更新玩家位置、生成玩家子弹、刷怪、推进敌人和子弹。
5. CollisionSystem.resolve() 处理子弹命中、敌人死亡、敌机撞击玩家、道具拾取。
6. 返回本帧获得的分数，并累加到 Game.score。
7. 若检测到玩家死亡，则停止计时器，播放失败音效，保存排行榜并绘制结算画面。
8. paintComponent() 调用 GameUI.render() 完成画面刷新。

核心流程可概括为：

$$\texttt{Main} \rightarrow \texttt{AppController} \rightarrow \texttt{Game} \rightarrow \texttt{DifficultyTemplate} \rightarrow \texttt{CharacterSystem} \rightarrow \texttt{CollisionSystem} \rightarrow \texttt{GameUI}$$

2.4 UML 建模与分析

本系统中最重要的核心实体包括英雄机、敌机、子弹与道具。它们都可以抽象为“游戏角色”或“可碰撞对象”。项目中使用 GameCharacter 作为基础父类，统一保存坐标、宽度、高度、速度、生命值、碰撞矩形和精灵资源。之后通过 Aircraft 扩展出具有发射子弹能力的飞行器，再由 Hero 和 Enemy 分别表示玩家与敌人。

该建模方式的优点是：

1. 将位置、尺寸、生命、碰撞矩形等公共属性上移到父类，减少重复代码。
2. 将“能够发射子弹”的能力抽象到 Aircraft，避免让道具、子弹等对象承担无关职责。
3. 将英雄机和敌机都视为飞行器，便于统一使用 FireStrategy 控制发射行为。
4. 将不同敌机和不同道具拆分为独立子类，便于扩展新的敌人与道具类型。

Listing 2: 核心实体 UML 类图

```
@startuml
skinparam classAttributeIconSize 0

package "components" {
  abstract class GameCharacter {
    -x: int
    -y: int
    -width: int
    -height: int
    -velocityX: int
    -velocityY: int
    -health: int
    +tick(): void
    +getBounds(): Rectangle
    +applyDamage(damage: int): void
    +heal(amount: int): void
    +isAlive(): boolean
    +getSpriteResource(): String
  }

  abstract class Aircraft {
    -fireStrategy: FireStrategy
```

```
  -projectileOwner: Projectile.Owner
  -projectileDamage: int
  -fireIntervalTicks: int
  -baseFireIntervalTicks: int
  +tryFire(): List<Projectile>
  +setFireStrategy(strategy: FireStrategy): void
  +setFireIntervalTicks(ticks: int): void
}

class Hero {
  -instance: Hero
  -defaultFireStrategy: FireStrategy
  -powerUpScheduler: ScheduledExecutorService
  -restoreFuture: ScheduledFuture
  +getInstance(x: int, y: int): Hero
  +resetInstance(): void
  +moveTo(x: int, y: int): void
  +applyHealthSupply(): void
  +applyFirepowerSupply(durationSeconds: int): void
  +applySuperFirepowerSupply(durationSeconds: int): void
  +restoreDefaultWeapon(): void
  +shutdown(): void
}

abstract class Enemy {
  -scoreValue: int
  -dropChance: double
  -dropCount: int
  -droppableItems: List<ItemType>
  +advance(width: int, height: int): void
  +freeze(ticks: int): void
  +slow(ticks: int, multiplier: double): void
  +isBoss(): boolean
  +getScoreValue(): int
}

class MobEnemy
class EliteEnemy
```

```
  class ElitePlusEnemy
  class AceEnemy
  class BossEnemy

  class Projectile {
    -owner: Owner
    -damage: int
    +fromPolar(...): Projectile
    +getOwner(): Owner
    +getDamage(): int
  }

  abstract class Item {
    -type: ItemType
    +activate(hero: Hero, context: ItemEffectContext): void
  }

  class HealthItem
  class FirepowerItem
  class SuperFirepowerItem
  class BombItem
  class FreezeItem

  enum ItemType {
    HEALTH
    FIREPOWER
    SUPER_FIREPOWER
    BOMB
    FREEZE
  }
}

GameCharacter <|-- Aircraft
Aircraft <|-- Hero
Aircraft <|-- Enemy
Enemy <|-- MobEnemy
Enemy <|-- EliteEnemy
Enemy <|-- ElitePlusEnemy
```

```
Enemy <|-- AceEnemy
Enemy <|-- BossEnemy

GameCharacter <|-- Projectile
GameCharacter <|-- Item
Item <|-- HealthItem
Item <|-- FirepowerItem
Item <|-- SuperFirepowerItem
Item <|-- BombItem
Item <|-- FreezeItem

Aircraft --> FireStrategy
Item --> ItemEffectContext
@enduml
```

3 设计模式应用

3.1 单例模式

3.1.1 应用场景分析

单例模式用于 Hero 英雄机对象。游戏运行过程中，玩家控制的英雄机在同一局游戏中只应存在一个。如果允许系统随意创建多个 Hero 实例，会导致输入控制、生命值、道具强化和碰撞检测的对象不一致。例如，鼠标移动可能控制一个英雄机，而碰撞系统判断的是另一个英雄机，从而破坏游戏状态一致性。

因此项目将 Hero 设计为单例类，通过静态字段 instance 保存唯一实例，通过 getInstance(x, y) 获取实例。开始新游戏时，系统调用 resetInstance() 释放旧实例并重新创建英雄机。这样既保证单局游戏中英雄机唯一，又允许多局游戏之间重置状态。

该设计的优势如下：

1. 保证全局唯一玩家对象，避免同一局游戏中出现多个英雄机。
2. 便于 Game、CharacterSystem、CollisionSystem 共享同一个英雄机引用。
3. 与道具强化机制配合较好，强化效果始终作用在唯一玩家对象上。
4. 通过 resetInstance() 支持重新开始游戏。

需要注意的是，单例模式会引入全局状态。为了避免影响测试和多局游戏重启，项目提供了 `resetInstance()` 与 `shutdown()`，用于清理英雄机的定时强化线程和旧状态。

3.1.2 设计模式结构图

Listing 3: Hero 单例模式结构图

```
@startuml
skinparam classAttributeIconSize 0

class Game {
  -hero: Hero
  +Game(difficulty: GameDifficulty, playerName: String)
  +tick(): void
}

class Hero {
  -static instance: Hero
  -defaultFireStrategy: FireStrategy
  -powerUpScheduler: ScheduledExecutorService
  -restoreFuture: ScheduledFuture
  -Hero(x: int, y: int, scheduler: ScheduledExecutorService, strategy:
      FireStrategy)
  +static getInstance(x: int, y: int): Hero
  +static resetInstance(): void
  +moveTo(x: int, y: int): void
  +applyHealthSupply(): void
  +applyFirepowerSupply(durationSeconds: int): void
  +applySuperFirepowerSupply(durationSeconds: int): void
  +shutdown(): void
}

Game --> Hero : obtains unique instance
@enduml
```

在该结构中，`Hero` 是 Singleton 角色，`instance` 是唯一实例，私有构造方法防止外部直接创建对象，`getInstance()` 是全局访问点，`resetInstance()` 用于游戏重启时释放旧对象。

3.2 简单工厂模式

3.2.1 应用场景分析

简单工厂模式用于道具创建。游戏中敌人死亡后可能掉落多种道具，例如回血、火力强化、超级火力、炸弹和冻结。外部模块不应关心每种道具的具体构造细节，只需要根据 `ItemType` 得到一个 `Item` 对象即可。

项目中的 `ItemFactory` 提供静态方法 `create(type, x, y)`，内部通过 `switch` 判断 `ItemType` 并创建对应子类。这样，`CollisionSystem` 在处理敌人死亡掉落时只需要调用 `ItemFactory.createDrops(enemy, random)`，而不需要知道各个道具类的构造细节。

该设计的优势如下：

1. 集中管理道具创建逻辑，降低碰撞系统与具体道具类之间的耦合。
2. 新增道具时只需扩展 `ItemType` 和 `ItemFactory`，调用方基本不变。
3. 道具掉落逻辑可以统一处理随机选择、掉落数量和掉落位置。

3.2.2 设计模式结构图

Listing 4: ItemFactory 简单工厂模式结构图

```
@startuml
skinparam classAttributeIconSize 0

enum ItemType {
  HEALTH
  FIREPOWER
  SUPER_FIREPOWER
  BOMB
  FREEZE
}

abstract class Item {
  -type: ItemType
  +activate(hero: Hero, context: ItemEffectContext): void
}

class HealthItem
class FirepowerItem
```

```
class SuperFirepowerItem
class BombItem
class FreezeItem

class ItemFactory {
  -ItemFactory()
  +static create(type: ItemType, x: int, y: int): Item
  +static createDrops(enemy: Enemy, random: Random): List<Item>
}

Item <|-- HealthItem
Item <|-- FirepowerItem
Item <|-- SuperFirepowerItem
Item <|-- BombItem
Item <|-- FreezeItem

ItemFactory ..> ItemType
ItemFactory ..> Item
ItemFactory ..> Enemy
@enduml
```

在该结构中，ItemFactory 是简单工厂角色，ItemType 是创建依据，Item 是抽象产品，HealthItem、FirepowerItem、SuperFirepowerItem、BombItem、FreezeItem 是具体产品。

3.3 工厂模式

3.3.1 应用场景分析

工厂模式用于敌机创建。游戏中敌机类型较多，包括普通敌人、精英敌人、强化精英敌人、王牌敌人和 Boss。不同敌人的尺寸、生命值、移动速度、分数、掉落规则和射击策略不同。如果在 CharacterSystem 中直接写大量 new MobEnemy()、new EliteEnemy() 等代码，会导致刷怪逻辑与敌机创建细节高度耦合。

项目通过 EnemyFactory 抽象敌机工厂接口或抽象类，并为不同敌人提供不同工厂实现，例如 MobEnemyFactory、EliteEnemyFactory、ElitePlusEnemyFactory、AceEnemyFactory、BossEnemyFactory。CharacterSystem 只保存工厂列表，并根据难度权重选择工厂生成敌人。

该设计的优势如下：

1. 将敌机创建细节从刷怪系统中分离出来。
2. 支持通过难度权重控制不同敌机出现概率。
3. 新增敌人类型时，只需要新增敌人类和对应工厂类，并加入工厂列表。
4. Boss 作为特殊敌人，也可使用独立工厂统一创建。

3.3.2 设计模式结构图

Listing 5: EnemyFactory 工厂模式结构图

```
@startuml
skinparam classAttributeIconSize 0

abstract class EnemyFactory {
  +createEnemy(random: Random, playfieldWidth: int): Enemy
}

class MobEnemyFactory {
  +createEnemy(random: Random, playfieldWidth: int): Enemy
}

class EliteEnemyFactory {
  +createEnemy(random: Random, playfieldWidth: int): Enemy
}

class ElitePlusEnemyFactory {
  +createEnemy(random: Random, playfieldWidth: int): Enemy
}

class AceEnemyFactory {
  +createEnemy(random: Random, playfieldWidth: int): Enemy
}

class BossEnemyFactory {
  +createEnemy(random: Random, playfieldWidth: int): Enemy
}

abstract class Enemy
class MobEnemy
```

```
class EliteEnemy
class ElitePlusEnemy
class AceEnemy
class BossEnemy

class CharacterSystem {
  -enemyFactories: List<EnemyFactory>
  -bossEnemyFactory: EnemyFactory
  -difficultyTemplate: DifficultyTemplate
  +onTick(width: int, height: int, score: int, tickCount: int): void
  -spawnEnemies(playfieldWidth: int, score: int): void
  -selectEnemyFactory(): EnemyFactory
}

EnemyFactory <|-- MobEnemyFactory
EnemyFactory <|-- EliteEnemyFactory
EnemyFactory <|-- ElitePlusEnemyFactory
EnemyFactory <|-- AceEnemyFactory
EnemyFactory <|-- BossEnemyFactory

Enemy <|-- MobEnemy
Enemy <|-- EliteEnemy
Enemy <|-- ElitePlusEnemy
Enemy <|-- AceEnemy
Enemy <|-- BossEnemy

MobEnemyFactory ..> MobEnemy
EliteEnemyFactory ..> EliteEnemy
ElitePlusEnemyFactory ..> ElitePlusEnemy
AceEnemyFactory ..> AceEnemy
BossEnemyFactory ..> BossEnemy

CharacterSystem --> EnemyFactory
CharacterSystem --> DifficultyTemplate
@enduml
```

在该结构中，EnemyFactory 是抽象工厂角色，具体工厂负责创建对应敌机，CharacterSystem 是客户端角色。客户端依赖工厂抽象，而不直接依赖具体敌人构

造。

3.4 策略模式

3.4.1 应用场景分析

策略模式用于子弹发射逻辑。飞机大战游戏中，英雄机和敌机都需要发射子弹，但不同角色的弹幕形态不同。例如，普通单发子弹、散射子弹、环形弹幕、扇形弹幕和直线阵列弹幕的生成算法完全不同。如果将所有发射逻辑都写在 `Aircraft.tryFire()` 中，会导致该方法不断膨胀，并出现大量条件判断。

项目通过 `FireStrategy` 抽象射击策略，`Aircraft` 持有一个 `FireStrategy` 对象。每当到达开火间隔，`Aircraft.tryFire()` 调用当前策略生成子弹列表。英雄机拾取火力道具后，只需要替换当前 `FireStrategy`，即可改变发射模式；强化结束后再恢复默认策略。

该设计的优势如下：

1. 将不同弹幕算法封装成独立策略类，避免条件判断堆积。
2. 飞机对象与具体弹幕算法解耦，运行时可以切换策略。
3. 道具强化效果可以通过替换策略自然实现。
4. 新增弹幕类型时，只需要新增 `FireStrategy` 实现类。

3.4.2 设计模式结构图

Listing 6: FireStrategy 策略模式结构图

```
@startuml
skinparam classAttributeIconSize 0

interface FireStrategy {
  +fire(aircraft: Aircraft): List<Projectile>
}

class SingleForwardFireStrategy
class SpreadShotStrategy
class RadialShotStrategy
class FanShotStrategy
class StraightRowFireStrategy

```

```
abstract class Aircraft {
  -fireStrategy: FireStrategy
  -fireIntervalTicks: int
  -fireCooldownTicks: int
  +tryFire(): List<Projectile>
  +setFireStrategy(strategy: FireStrategy): void
}

class Hero {
  -defaultFireStrategy: FireStrategy
  +applyFirepowerSupply(durationSeconds: int): void
  +applySuperFirepowerSupply(durationSeconds: int): void
  +restoreDefaultWeapon(): void
}

class Projectile

FireStrategy <|.. SingleForwardFireStrategy
FireStrategy <|.. SpreadShotStrategy
FireStrategy <|.. RadialShotStrategy
FireStrategy <|.. FanShotStrategy
FireStrategy <|.. StraightRowFireStrategy

Aircraft --> FireStrategy
Aircraft <|-- Hero
FireStrategy ..> Projectile
@enduml
```

在该结构中，FireStrategy 是策略接口，具体策略类实现不同子弹生成算法，Aircraft 是上下文角色，Hero 通过修改策略实现临时强化。

3.5 数据访问对象模式

3.5.1 应用场景分析

数据访问对象模式用于排行榜模块。游戏结束后，系统需要保存玩家名、分数和游玩时间，并在排行榜界面展示历史记录。如果游戏逻辑直接使用文件读写 API，会导致游戏核心逻辑与存储格式耦合，后续若从 TSV 文件改为数据库、JSON 或网络接口，需要修改大量业务代码。

项目定义 LeaderboardDao 接口，将排行榜的保存、查询和删除操作抽象为数据访问接口；具体类 FileLeaderboardDao 使用本地 TSV 文件实现该接口。这样，Game 和 LeaderboardPanel 只依赖 LeaderboardDao 抽象，不依赖具体文件格式。

该设计的优势如下：

1. 将数据持久化逻辑从游戏核心逻辑中分离。
2. 隐藏文件路径、编码、排序和异常处理细节。
3. 为后续替换存储方式提供扩展点。
4. 便于测试时注入内存版或伪造版 DAO。

3.5.2 设计模式结构图

Listing 7: Leaderboard DAO 模式结构图

```
@startuml
skinparam classAttributeIconSize 0

interface LeaderboardDao {
  +save(difficulty: GameDifficulty, entry: LeaderboardEntry): void
  +findAll(difficulty: GameDifficulty): List<LeaderboardEntry>
  +delete(difficulty: GameDifficulty, entry: LeaderboardEntry): void
}

class FileLeaderboardDao {
  -baseDirectory: Path
  +save(difficulty: GameDifficulty, entry: LeaderboardEntry): void
  +findAll(difficulty: GameDifficulty): List<LeaderboardEntry>
  +delete(difficulty: GameDifficulty, entry: LeaderboardEntry): void
  -resolveStoragePath(difficulty: GameDifficulty): Path
  -sanitize(playerName: String): String
}

record LeaderboardEntry {
  playerName: String
  score: int
  playedAt: LocalDateTime
}

```

```
enum GameDifficulty {
  EASY
  NORMAL
  HARD
}

class Game {
  -leaderboardDao: LeaderboardDao
  -leaderboardEntries: List<LeaderboardEntry>
  -handleGameOver(): void
}

class LeaderboardPanel {
  -leaderboardDao: LeaderboardDao
  +refresh(): void
}

LeaderboardDao <|.. FileLeaderboardDao
FileLeaderboardDao ..> LeaderboardEntry
FileLeaderboardDao ..> GameDifficulty
Game --> LeaderboardDao
LeaderboardPanel --> LeaderboardDao
@enduml
```

在该结构中，`LeaderboardDao` 是 DAO 抽象，`FileLeaderboardDao` 是具体数据访问对象，`LeaderboardEntry` 是数据记录对象，`Game` 和 `LeaderboardPanel` 是 DAO 的调用方。

3.6 观察者模式

3.6.1 应用场景分析

观察者模式用于道具强化事件。某些道具的效果不仅影响玩家本身，还会影响敌人、子弹、音效等多个模块。例如冻结道具可能使敌人减速或冻结，同时需要播放强化音效；炸弹道具可能清除敌方对象并产生声音反馈。如果让道具类直接依赖所有受影响模块，会导致道具类与敌人列表、子弹列表、音频系统高度耦合。

项目使用 `PowerUpDispatcher` 作为事件分发器，维护观察者列表。当道具被激活时，通过 `notifyObservers(event)` 广播强化事件。不同观察者根据事件类型执行自

己的响应逻辑，例如敌人强化观察者处理敌人与子弹状态，音频强化观察者处理音效。

该设计的优势如下：

1. 道具激活者不需要知道所有响应方。
2. 新增道具响应行为时，只需新增观察者，不必修改道具类。
3. 可以将敌人控制、音频反馈等横切逻辑分离。
4. 使用事件对象统一传递强化类型和上下文信息。

3.6.2 设计模式结构图

Listing 8: PowerUp 观察者模式结构图

```
@startuml
skinparam classAttributeIconSize 0

class PowerUpDispatcher {
  -observers: List<PowerUpObserver>
  +addObserver(observer: PowerUpObserver): void
  +removeObserver(observer: PowerUpObserver): void
  +notifyObservers(event: PowerUpEvent): void
}

interface PowerUpObserver {
  +onPowerUpActivated(event: PowerUpEvent): void
}

class EnemyPowerUpObserver {
  -enemies: List<Enemy>
  -projectiles: List<Projectile>
  -freezeDurationTicks: int
  +onPowerUpActivated(event: PowerUpEvent): void
}

class AudioPowerUpObserver {
  -audioManager: AudioManager
  +onPowerUpActivated(event: PowerUpEvent): void
}

```

```
class PowerUpEvent {
  -type: ItemType
  -hero: Hero
}

class ItemEffectContext {
  -audioManager: AudioManager
  -difficultyTemplate: DifficultyTemplate
  -powerUpDispatcher: PowerUpDispatcher
}

abstract class Item {
  +activate(hero: Hero, context: ItemEffectContext): void
}

PowerUpObserver <|.. EnemyPowerUpObserver
PowerUpObserver <|.. AudioPowerUpObserver
PowerUpDispatcher o--> PowerUpObserver
PowerUpDispatcher ..> PowerUpEvent
Item --> ItemEffectContext
ItemEffectContext --> PowerUpDispatcher
@enduml
```

在该结构中，PowerUpDispatcher 是主题或发布者，PowerUpObserver 是观察者接口，EnemyPowerUpObserver 与 AudioPowerUpObserver 是具体观察者，PowerUpEvent 是事件对象。

3.7 模板模式

3.7.1 应用场景分析

模板模式用于难度系统。不同难度的游戏流程基本相同：先根据难度调整参数，再推进角色和子弹，然后进行碰撞检测，最后执行碰撞后的附加处理。但不同难度下的具体参数不同，例如敌人生成间隔、敌人数量上限、Boss 生成阈值、敌人速度、血量、射击间隔、玩家射击间隔和成长机制。

如果将简单、普通、困难难度分别写成三套完整主循环，会产生大量重复代码，并且不同难度之间的流程可能逐渐不一致。项目使用抽象类 DifficultyTemplate 固定主流程，由子类重写若干钩子方法调整参数。这样既保证所有难度执行一致的主流

程，又允许不同难度提供不同规则。

该设计的优势如下：

1. 固定游戏循环骨架，避免不同难度流程分裂。
2. 将难度差异集中在可重写的参数方法中。
3. 新增难度时只需继承模板类并覆盖相关方法。
4. 易于维护游戏平衡性，例如单独调节刷怪、Boss、速度和血量。

3.7.2 设计模式结构图

Listing 9: DifficultyTemplate 模板方法模式结构图

```
@startuml
skinparam classAttributeIconSize 0

abstract class DifficultyTemplate {
  +final action(context: GameLoopContext): int
  +final enemySpawnIntervalTicks(tickCount: int): int
  +final configureEnemy(enemy: Enemy, tickCount: int, bossSpawnCount: int): void
  +final shouldSpawnBoss(bossPresent: boolean, score: int, threshold: int): boolean
  #baseEnemySpawnIntervalTicks(): int
  #baseMaxNonBossEnemies(): int
  #baseInitialBossScoreThreshold(): int
  #enemySpeedMultiplier(tickCount: int): double
  #enemyHealthIncrease(tickCount: int): int
  #heroFireIntervalTicks(baseFireIntervalTicks: int, tickCount: int): int
  #enemyFireIntervalTicks(baseFireIntervalTicks: int, tickCount: int): int
  #afterCollisions(context: GameLoopContext, scoreGained: int): void
}

class EasyDifficultyTemplate
class NormalDifficultyTemplate
class HardDifficultyTemplate

class GameLoopContext {
  +game(): Game
```

```
  +characterSystem(): CharacterSystem
  +collisionSystem(): CollisionSystem
  +tickCount(): int
  +score(): int
}

class Game {
  -difficultyTemplate: DifficultyTemplate
  +tick(): void
}

DifficultyTemplate <|-- EasyDifficultyTemplate
DifficultyTemplate <|-- NormalDifficultyTemplate
DifficultyTemplate <|-- HardDifficultyTemplate

Game --> DifficultyTemplate
DifficultyTemplate --> GameLoopContext
GameLoopContext --> CharacterSystem
GameLoopContext --> CollisionSystem
@enduml
```

在该结构中，DifficultyTemplate 是抽象模板类，action() 是模板方法；简单、普通和困难难度模板是具体实现类。

4 系统亮点与创新

4.1 多设计模式组合

本项目不是孤立地使用某一个设计模式，而是围绕完整游戏流程组合使用多种设计模式。英雄机唯一性使用单例模式；敌人与道具创建使用工厂相关模式；子弹发射使用策略模式；排行榜持久化使用 DAO 模式；道具事件使用观察者模式；难度流程使用模板方法模式。这种组合体现了面向对象设计中“高内聚、低耦合”的思想。

4.2 弹幕系统具有较强扩展性

项目将弹幕生成抽象为 FireStrategy。因此，新增一种弹幕形态不需要修改英雄机或敌机类，只需要新增一个策略类。例如可以继续扩展追踪弹、螺旋弹、交叉弹幕或分裂弹幕。这种结构使游戏玩法扩展成本较低。

4.3 难度系统层次清晰

难度系统没有简单地使用大量 `if-else` 判断，而是使用模板方法，将统一流程和差异参数分开。简单难度可以降低敌人强度，普通难度可以随时间推进，困难难度可以提高敌人数量和 Boss 压力。该设计有利于后续加入“无尽模式”“挑战模式”等新模式。

4.4 排行榜实现简单可靠

排行榜采用本地 TSV 文件持久化，每个难度对应独立文件。相比数据库方案，该方式依赖少、实现简单，适合作为课程项目的数据持久化方案。同时项目通过 DAO 接口隐藏文件实现细节，保留了未来替换存储方式的可能性。

4.5 游戏反馈完整

系统包含背景音乐、Boss 音乐切换、子弹命中音效、游戏结束音效和静音控制。游戏还具有 HUD、分数、生命值、排行榜等反馈信息，使最终产出不仅是逻辑演示，而是一个可交互、可运行、较完整的桌面小游戏。

5 收获与反思

5.1 核心收获

通过本次实验，我对面向对象设计在实际项目中的作用有了更直观的理解。飞机大战游戏虽然规模不大，但其内部已经包含角色建模、对象创建、状态更新、碰撞检测、事件响应、数据持久化、UI 渲染等多个方面。如果没有良好的结构划分，所有逻辑很容易集中在一个庞大的类中，导致后续维护困难。

本项目通过 `components`、`core`、`strategy`、`factories`、`difficulty`、`observer`、`leaderboard` 等包拆分职责，使系统边界较为清晰。例如，`CharacterSystem` 不负责绘图，`GameUI` 不负责碰撞，`FileLeaderboardDao` 不负责游戏规则，`FireStrategy` 不负责角色生命周期。这种设计体现了单一职责原则。

同时，本次实验也加深了我对设计模式的理解。设计模式并不是为了“套模板”，而是为了解决具体问题。单例模式解决英雄机唯一性问题；策略模式解决弹幕算法可替换问题；工厂模式解决敌人创建解耦问题；观察者模式解决道具事件多方响应问题；模板方法模式解决难度流程统一但参数不同的问题。每个模式都对应一个真实的变化点。

5.2 遇到的问题与解决方案

5.2.1 问题一：游戏主循环容易变成上帝类

游戏项目中，Game 很容易承担所有职责，包括输入、移动、刷怪、碰撞、绘图、音频、排行榜等。如果全部写在 Game 中，类会迅速膨胀。

解决方案是将逻辑拆分为多个系统类：CharacterSystem 负责角色推进和刷怪，CollisionSystem 负责碰撞结算，GameUI 负责画面绘制，LeaderboardDao 负责数据持久化。这样 Game 主要作为协调者，而不是所有细节的实现者。

5.2.2 问题二：弹幕逻辑变化频繁

游戏中不同敌人和道具都会改变子弹形态。如果把弹幕逻辑写死在角色类中，每新增一种子弹形态都要修改角色类。

解决方案是将弹幕逻辑抽象为 FireStrategy。角色只负责在合适时机调用策略，具体如何生成子弹由策略对象决定。道具强化时只需要替换策略对象，强化结束后恢复默认策略。

5.2.3 问题三：不同难度之间重复代码较多

简单、普通、困难难度的流程相同，但参数不同。如果复制三份主循环，会产生大量重复代码。

解决方案是使用模板方法模式。DifficultyTemplate.action() 固定主流程，具体难度通过覆盖参数方法改变行为。例如敌人生成间隔、敌人速度、Boss 阈值和火力间隔都可以通过子类方法调整。

5.2.4 问题四：排行榜持久化与游戏逻辑耦合

如果游戏结束时直接在 Game 中写文件，会导致游戏逻辑依赖具体文件格式，后续很难扩展。

解决方案是定义 LeaderboardDao 接口，并由 FileLeaderboardDao 实现本地文件存储。游戏逻辑只依赖接口，具体存储细节由 DAO 实现。

5.3 不足与改进方向

虽然项目结构较完整，但仍有一些可以改进的地方：

1. **进一步降低 Game 的协调压力**。当前 Game 仍然持有较多对象并负责较多流程控制。后续可以引入更明确的 GameState 或 SceneManager。

2. **增强配置化能力**。敌人属性、道具掉落率、难度参数目前主要写在代码中。可以改为 JSON、YAML 或 properties 配置文件，便于调参。
3. **提升资源加载健壮性**。图片和音频资源加载失败时，可以提供默认资源、错误日志和用户提示。
4. **改进单例对测试的影响**。`Hero` 单例虽然保证唯一性，但会带来全局状态。后续可考虑由 `GameContext` 管理唯一英雄机，而不是静态单例。
5. **完善自动化测试**。可以增加碰撞检测、排行榜排序、道具触发、难度参数变化和策略生成子弹数量等单元测试。
6. **进一步分离 UI 与逻辑**。当前项目基于 Swing，逻辑与绘制之间仍存在一定耦合。后续可尝试 MVC 或 ECS 思路。

6 参考资料

1. PlantUML 官方文档：https://plantuml.com/
2. Oracle Java Swing 文档：https://docs.oracle.com/javase/tutorial/uiswing/

A 附录：总体架构 UML 类图

Listing 10: Touhou 项目总体架构类图

```
@startuml
skinparam classAttributeIconSize 0

package "com.touhou" {
  class Main {
    +main(args: String[]): void
    +createGame(): Game
    +createGame(difficulty: GameDifficulty, playerName: String): Game
  }
}

package "ui" {
  class Layout {
    +WINDOW_WIDTH: int
```

```
    +WINDOW_HEIGHT: int
    +showPanel(panel: JPanel): void
  }

  class AppController {
    -layout: Layout
    -leaderboardDao: LeaderboardDao
    -audioManager: AudioManager
    +showMenu(): void
    +showLeaderboard(difficulty: GameDifficulty): void
    +startGame(difficulty: GameDifficulty, playerName: String): void
  }

  class MenuPanel
  class LeaderboardPanel
  class GameUI {
    +render(...): void
  }
}

package "core" {
  class Game {
    +TICKS_PER_SECOND: int
    -hero: Hero
    -enemies: List<Enemy>
    -projectiles: List<Projectile>
    -items: List<Item>
    -characterSystem: CharacterSystem
    -collisionSystem: CollisionSystem
    -gameUI: GameUI
    -difficultyTemplate: DifficultyTemplate
    -leaderboardDao: LeaderboardDao
    -audioManager: AudioManager
    +start(): void
    +stop(): void
    +tick(): void
    +getScore(): int
  }
```

```
  class CharacterSystem {
    +onTick(width: int, height: int, score: int, tickCount: int): void
  }

  class CollisionSystem {
    +overlaps(left: GameCharacter, right: GameCharacter): boolean
    +resolve(hero: Hero, enemies: List<Enemy>, projectiles: List<Projectile>,
        items: List<Item>): int
  }

  class ItemEffectContext
}

package "components" {
  abstract class GameCharacter
  abstract class Aircraft
  class Hero
  abstract class Enemy
  class Projectile
  abstract class Item
  enum ItemType
}

package "strategy" {
  interface FireStrategy {
    +fire(aircraft: Aircraft): List<Projectile>
  }

  class SingleForwardFireStrategy
  class SpreadShotStrategy
  class RadialShotStrategy
  class FanShotStrategy
  class StraightRowFireStrategy
}

package "difficulty" {
  abstract class DifficultyTemplate {
```

```
    +action(context: GameLoopContext): int
    +configureEnemy(enemy: Enemy, tickCount: int, bossSpawnCount: int): void
    +shouldSpawnBoss(...): boolean
  }

  class EasyDifficultyTemplate
  class NormalDifficultyTemplate
  class HardDifficultyTemplate
  class GameLoopContext
}

package "factories" {
  abstract class EnemyFactory {
    +createEnemy(random: Random, playfieldWidth: int): Enemy
  }

  class MobEnemyFactory
  class EliteEnemyFactory
  class ElitePlusEnemyFactory
  class AceEnemyFactory
  class BossEnemyFactory

  class ItemFactory {
    +create(type: ItemType, x: int, y: int): Item
    +createDrops(enemy: Enemy, random: Random): List<Item>
  }
}

package "leaderboard" {
  interface LeaderboardDao {
    +save(difficulty: GameDifficulty, entry: LeaderboardEntry): void
    +findAll(difficulty: GameDifficulty): List<LeaderboardEntry>
    +delete(difficulty: GameDifficulty, entry: LeaderboardEntry): void
  }

  class FileLeaderboardDao
  record LeaderboardEntry
  enum GameDifficulty
```

```
  class LeaderboardPresenter
}

package "observer" {
  class PowerUpDispatcher
  interface PowerUpObserver
  class EnemyPowerUpObserver
  class AudioPowerUpObserver
  class PowerUpEvent
}

package "audio" {
  class AudioManager {
    +playEffect(resourcePath: String): void
    +playBackgroundLoop(resourcePath: String): void
    +stopBackground(): void
    +toggleMuted(): void
  }
}

GameCharacter <|-- Aircraft
Aircraft <|-- Hero
Aircraft <|-- Enemy
GameCharacter <|-- Projectile
GameCharacter <|-- Item

FireStrategy <|.. SingleForwardFireStrategy
FireStrategy <|.. SpreadShotStrategy
FireStrategy <|.. RadialShotStrategy
FireStrategy <|.. FanShotStrategy
FireStrategy <|.. StraightRowFireStrategy
Aircraft --> FireStrategy

DifficultyTemplate <|-- EasyDifficultyTemplate
DifficultyTemplate <|-- NormalDifficultyTemplate
DifficultyTemplate <|-- HardDifficultyTemplate

EnemyFactory <|-- MobEnemyFactory
```

```
EnemyFactory <|-- EliteEnemyFactory
EnemyFactory <|-- ElitePlusEnemyFactory
EnemyFactory <|-- AceEnemyFactory
EnemyFactory <|-- BossEnemyFactory

LeaderboardDao <|.. FileLeaderboardDao
PowerUpObserver <|.. EnemyPowerUpObserver
PowerUpObserver <|.. AudioPowerUpObserver
PowerUpDispatcher o--> PowerUpObserver

Main --> AppController
AppController --> Layout
AppController --> Game
AppController --> LeaderboardDao
AppController --> AudioManager

Game --> CharacterSystem
Game --> CollisionSystem
Game --> GameUI
Game --> DifficultyTemplate
Game --> LeaderboardDao
Game --> AudioManager
Game --> Hero
Game --> Enemy
Game --> Projectile
Game --> Item

CharacterSystem --> EnemyFactory
CollisionSystem --> ItemFactory
Item --> ItemEffectContext
GameUI --> GameCharacter

@enduml
```